



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Yulia Verigina
01 June 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

❑ Summary of methodologies

- Data Collection through API
- Data Collection with Web Scraping
- Data Wrangling
- Exploratory Data Analysis with SQL
- Exploratory Data Analysis with Data Visualization
- Machine Learning Prediction

❑ Summary of all results

- Exploratory Data Analysis Result
- Interactive Analytics in Screenshots
- Predictive Analytics Result

Introduction

➤ Project background and context

SpaceX has disrupted the aerospace industry by advertising Falcon 9 rocket launches at a highly competitive \$62 million each. These substantial cost savings are driven primarily by the company's ability to reuse the rocket's first stage. By determining whether this first stage will land successfully, we can accurately estimate the actual cost of a launch—critical intelligence for any competitor looking to bid effectively against SpaceX. To address this business need, this project develops a robust machine learning pipeline designed to predict first-stage landing success, providing the data-driven insights necessary to optimize competitive bidding strategies.

➤ Problems you want to find answers

- How can we accurately predict whether a Falcon 9 first stage will land successfully?
- How does a successful or failed landing quantitatively impact the actual cost of a launch?
- How can a competing aerospace company use these predictive insights to structure a winning commercial bid?

Section 1

Methodology

Methodology

Executive Summary

➤ **Data collection methodology**

- The training data was compiled using a combination of structured data from the SpaceX API and unstructured text scraped from Wikipedia. Describe how data was collected

➤ **Perform data wrangling**

- We converted text-based categories (like launch sites and orbit types) into a binary numerical format so the machine learning algorithms could process them accurately.

➤ **Perform exploratory data analysis (EDA) using visualization and SQL**

➤ **Perform interactive visual analytics using Folium and Plotly Dash**

➤ **Perform predictive analysis using classification models**

- Developed a predictive machine learning model to classify the landing outcomes of Falcon 9 first-stage boosters.

Data Collection

➤ **To build a reliable training dataset, we executed a two-pronged data collection and engineering pipeline:**

- API Integration: Retrieved official flight logs by sending GET requests to the SpaceX API.
- Data Parsing: Decoded the JSON payloads and flattened the nested structures into a structured Pandas DataFrame using `json_normalize()`.
- Data Cleansing: Inspected the dataset for missing values, applying targeted imputation and cleaning techniques to ensure data integrity.
- Web Scraping: Extracted complementary Falcon 9 launch records from Wikipedia using BeautifulSoup.
- HTML Table Extraction: Parsed raw HTML tables from the web pages and converted them into a standardized Pandas DataFrame for downstream analysis.

Data Collection – SpaceX API

- The dataset was compiled by executing GET requests against the SpaceX API. Subsequent preprocessing involved data cleaning, wrangling, and standardized formatting.
- Notebook link:
<https://github.com/ZombieYulZ/saay/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>

Executed GET requests to retrieve rocket launch data via the API.

```
[8]: spacex_url="https://api.spacexdata.com/v4/launches/past"
[9]: response = requests.get(spacex_url)
```

Utilized `json_normalize` to transform the semi-structured `JSON` response into a structured `DataFrame`.

Now we decode the response content as a `json()`, and turn it into a `Pandas dataframe` using `.json_normalize()`

```
[10]: # Use json_normalize method to convert the json result into a dataframe
data=pd.json_normalize(response.json())
```

We then cleaned the `dataset` and imputed the missing values.

```
[52]: # Calculate the mean value of PayloadMass column
payload_mean=data_falcon9['PayloadMass'].mean()
|
# Replace the np.nan values with its mean value
data_falcon9['PayloadMass']=data_falcon9['PayloadMass'].replace(np.nan,payload_mean)
print(f"Calculated Mean: {payload_mean}")
print(data_falcon9[['PayloadMass']].head())
```

Data Collection - Scraping

- We scraped the Falcon 9 launch records using BeautifulSoup, parsed the HTML table, and converted the data into a Pandas DataFrame.
- Notebook link:
[https://github.com/ZombieYulZ/saay/blob/main/jupyter-labs-webscraping%20\(1\).ipynb](https://github.com/ZombieYulZ/saay/blob/main/jupyter-labs-webscraping%20(1).ipynb)

To get our raw data, we applied an HTTP GET request to pull down all the Falcon 9 launch records.

```
[5]: static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                "AppleWebKit/537.36 (KHTML, like Gecko) "
                "Chrome/91.0.4472.124 Safari/537.36"
}
```

```
[6]: # use requests.get() method with the provided static_url and headers
# assign the response to a object
response=requests.get(static_url, headers=headers)
html_context=response.text
```

Once we got the HTML response back, we passed it into BeautifulSoup to start parsing the text.

```
[7]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup=BeautifulSoup(response.text,'html.parser')
```

Print the page title to verify if the BeautifulSoup object was created properly

```
[8]: # Use soup.title attribute
print(soup.title.string)

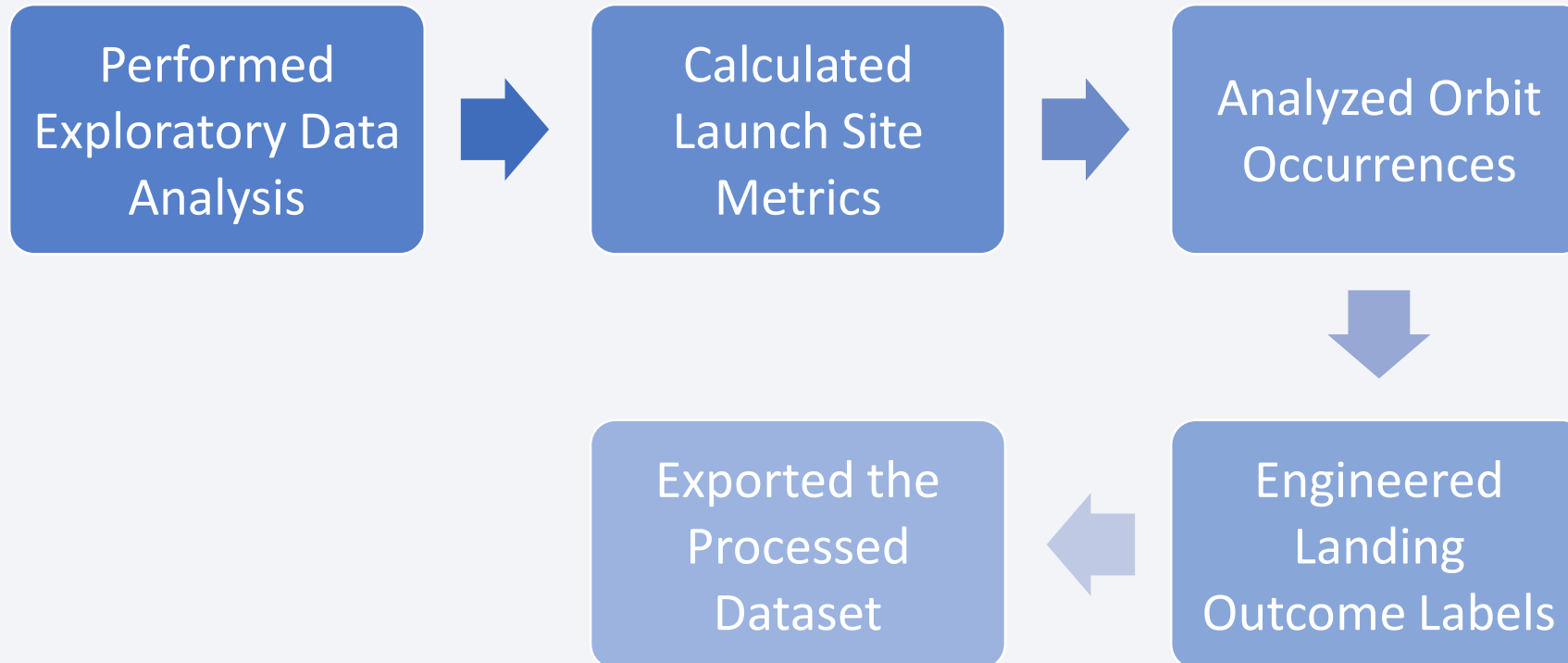
List of Falcon 9 and Falcon Heavy launches - Wikipedia
```

Next, we targeted the header elements to extract the column names so we could build a clean data table.

```
[11]: column_names = []
for th in first_launch_table.find_all('th'):
    name=extract_column_from_header(th)
    if name is not None and len(name)>0:
        column_names.append(name)
```

Next, we took those web tables, extracted the rows, and loaded everything into a structured DataFrame for analysis and export it to csv.

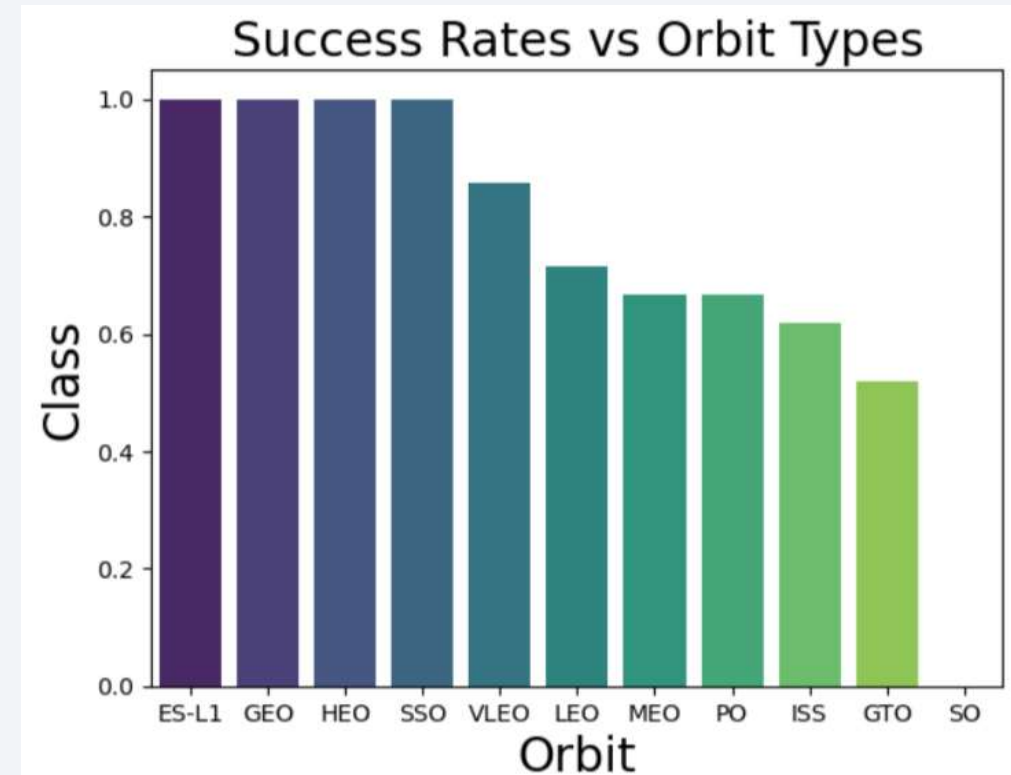
Data Wrangling



- Notebook Link: [https://github.com/ZombieYulZ/saay/blob/main/labs-jupyter-spacex-Data%20wrangling%20\(1\).ipynb](https://github.com/ZombieYulZ/saay/blob/main/labs-jupyter-spacex-Data%20wrangling%20(1).ipynb)

EDA with Data Visualization

- Data exploration was conducted by visualizing the relationship between flight numbers and launch site success rates across different orbit types. Additionally, we modeled the interplay between flight counts and orbit configurations, alongside the evaluation of yearly success trends.



- Notebook link:
[https://github.com/ZombieYulZ/saay/blob/main/edadataviz%20\(2\).ipynb](https://github.com/ZombieYulZ/saay/blob/main/edadataviz%20(2).ipynb)

EDA with SQL

- **The SpaceX dataset was loaded directly into a PostgreSQL database instance, bypassing the local Jupyter Notebook runtime environment.**
- **We executed targeted SQL queries to uncover critical data trends, isolating:**
 - Launch Locations: Identified all unique space mission launch sites.
 - NASA Core Payloads: Calculated total booster payload mass for NASA (CRS) missions.
 - Booster Performance: Computed the average payload mass for the F9 v1.1 version.
 - Mission Metrics: Aggregated total successful and failed mission outcomes.
 - Anomaly Tracking: Isolated drone ship landing failures alongside their booster versions and launch locations.
- **Notebook link:** [https://github.com/ZombieYulZ/saay/blob/main/jupyter-labs-eda-sql-coursera_sqlite%20\(2\).ipynb](https://github.com/ZombieYulZ/saay/blob/main/jupyter-labs-eda-sql-coursera_sqlite%20(2).ipynb)

Build an Interactive Map with Folium

- All historical launch sites were visualized on a Folium map using layered map objects. Circular regions and coordinate markers tracked the distribution of launch outcomes, while linear segments calculated site proximities to key geographic features.
- The categorical launch outcomes were converted into a binary target variable, where a value of 0 denotes a landing failure and 1 represents a successful landing.
- By analyzing the color-coded marker clusters, we successfully identified which specific launch sites exhibited a significantly higher operational success rate.
- We performed a proximity analysis to calculate the spatial distances between each launch site and its surrounding infrastructure. Through this process, we evaluated key operational parameters, such as whether launch pads are positioned near railways, highways, and coastlines, and if they maintain a safe buffer distance from heavily populated cities.
- Notebook Link:
https://github.com/ZombieYulZ/saay/blob/main/lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

- Interactive Dashboard: Developed an analytical dashboard using Plotly Dash.
- Distribution Analysis: Implemented pie charts to visualize total launch volumes across specific sites.
- Correlation Tracking: Deployed scatter plots mapping landing outcomes against payload masses across different booster versions.
- Notebook Link: [https://github.com/ZombieYulZ/saay/blob/main/spacex-dash-app%20\(10\).py](https://github.com/ZombieYulZ/saay/blob/main/spacex-dash-app%20(10).py)

Predictive Analysis (Classification)

- Data Ingestion & Splitting: Loaded telemetry via NumPy and Pandas, transforming features prior to a train/test split.
- Hyperparameter Optimization: Evaluated multiple classification models, leveraging GridSearchCV for systematic tuning.
- Iterative Refinement: Maximized accuracy metrics through strategic feature engineering and algorithm optimization.
- Model Selection: Identified the peak-performing classification model based on test-set accuracy scores.
- Notebook Link:
[https://github.com/ZombieYulZ/saay/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20\(1\).ipynb](https://github.com/ZombieYulZ/saay/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5%20(1).ipynb)

Results

➤ **Key Findings: SpaceX Falcon 9 Exploratory Data Analysis Launch Site Variation:**

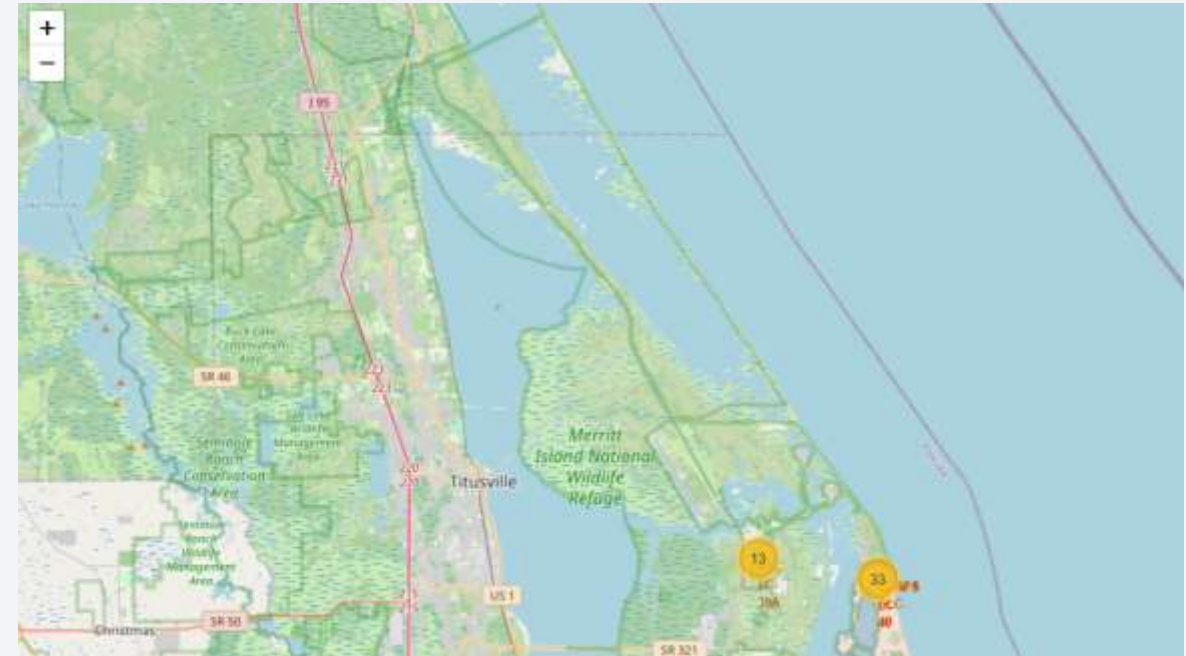
- SpaceX utilizes multiple diverse launch sites to accommodate different mission profiles and orbital trajectories.
- Early Customer Base: The initial Falcon 9 launches were primarily executed for SpaceX's internal test objectives and NASA resupply contracts.
- Baseline Payload Performance: The Falcon 9 v1.1 booster variant carried an average payload mass of 2,928 kg.
- Landing Milestones: SpaceX achieved its first historic successful booster recovery in 2015, exactly five years after the maiden Falcon 9 launch.
- Drone Ship Success Factors: High-payload missions (exceeding the 2,928 kg average) demonstrated high landing success rates when recovered on autonomous spaceport drone ships (ASDS).
- High Overall Reliability: The overall mission success rate for Falcon 9 flights approaches 100%.
- Early Recovery Anomalies: Drone ship landing attempts encountered challenges in 2015, resulting in failed recovery attempts for boosters F9 v1.1 B1012 and F9 v1.1 B1015.
- Temporal Improvement: Booster landing recovery rates showed a clear, iterative trajectory of improvement year-over-year as the technology matured.

Results

➤ Interactive analytics revealed that launch sites are typically located in safe coastal areas with strong logistics infrastructure. Most launches take place from east coast facilities.

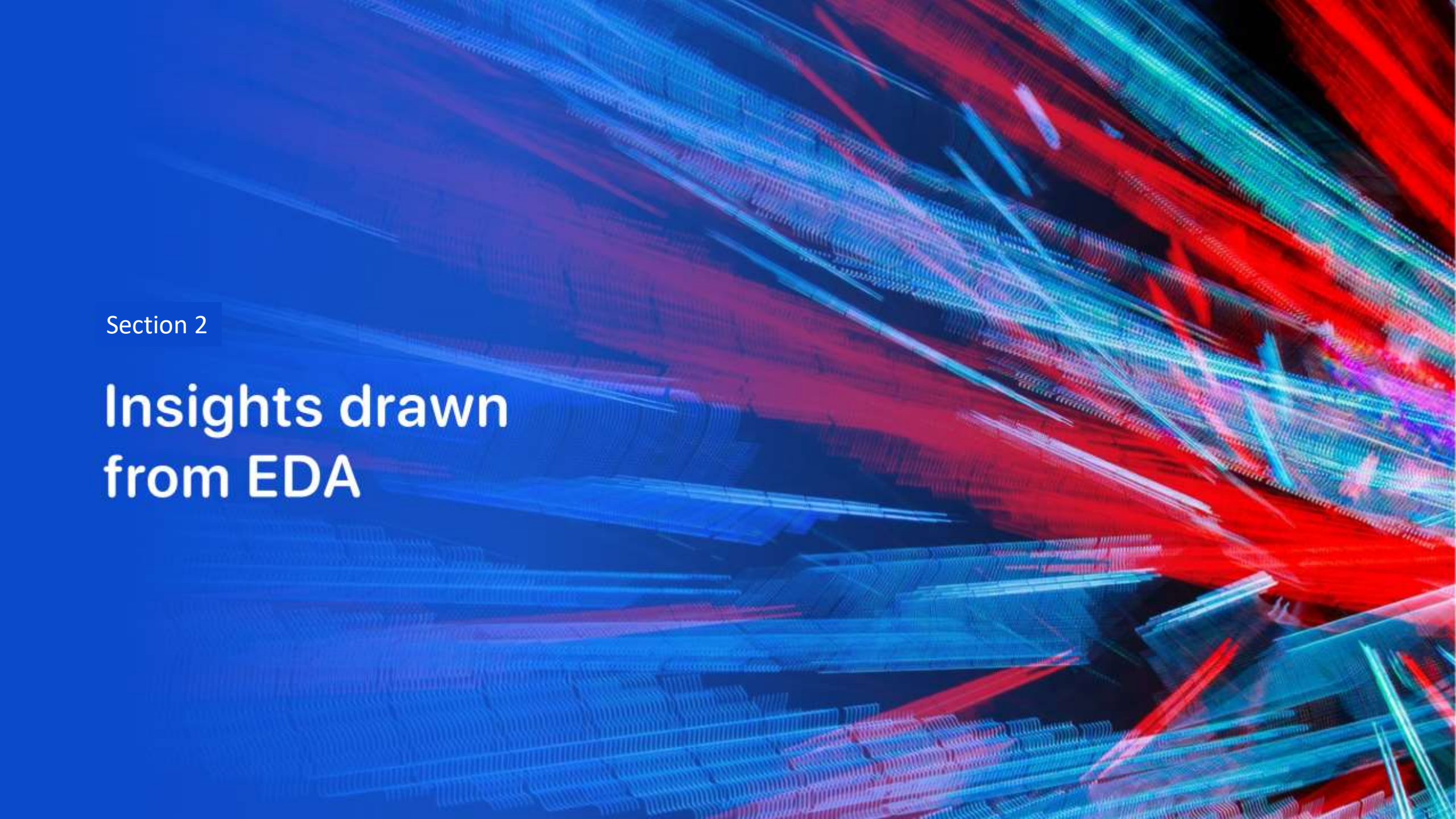
➤ Key Insights Identified

- Strategic Locations: Sites sit near oceans to ensure public safety.
- Robust Infrastructure: Locations feature highly developed logistical transport networks.
- Geographic Preference: The majority of liftoffs occur on the east coast.



Results

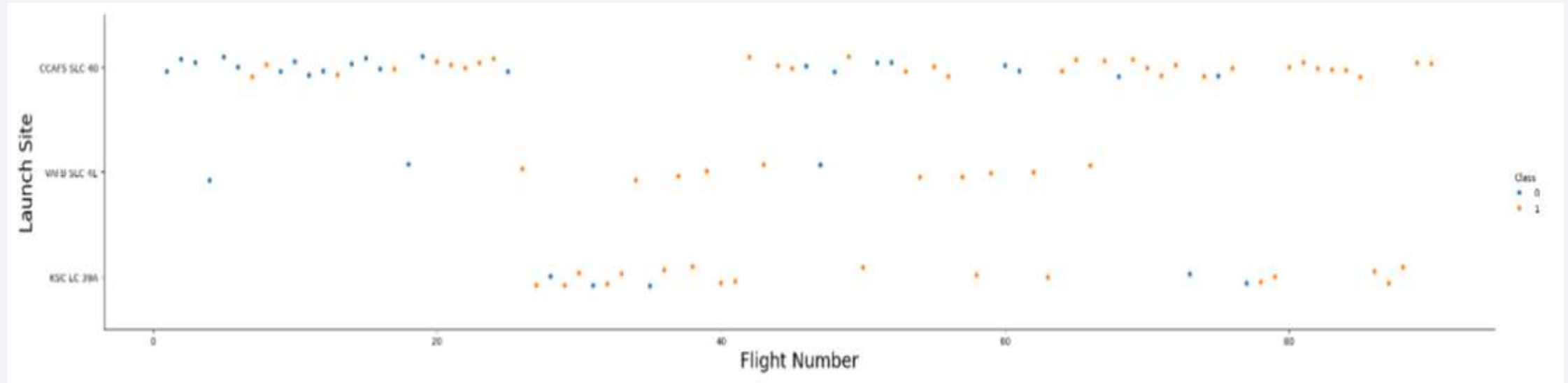
- Data Processing: Loaded, cleaned, and transformed telemetry records using NumPy and Pandas.
- Data Splitting: Partitioned the cleaned dataset into distinct training sets and testing matrices (random_state=42).
- Model Training: Developed multiple predictive algorithms using various classification models.
- Hyperparameter Tuning: Utilized GridSearchCV to systematically optimize model parameters (e.g., constraining max_depth to prevent overfitting).
- Performance Evaluation: Targeted Accuracy as the core metric to identify the best-performing classification model.



Section 2

Insights drawn from EDA

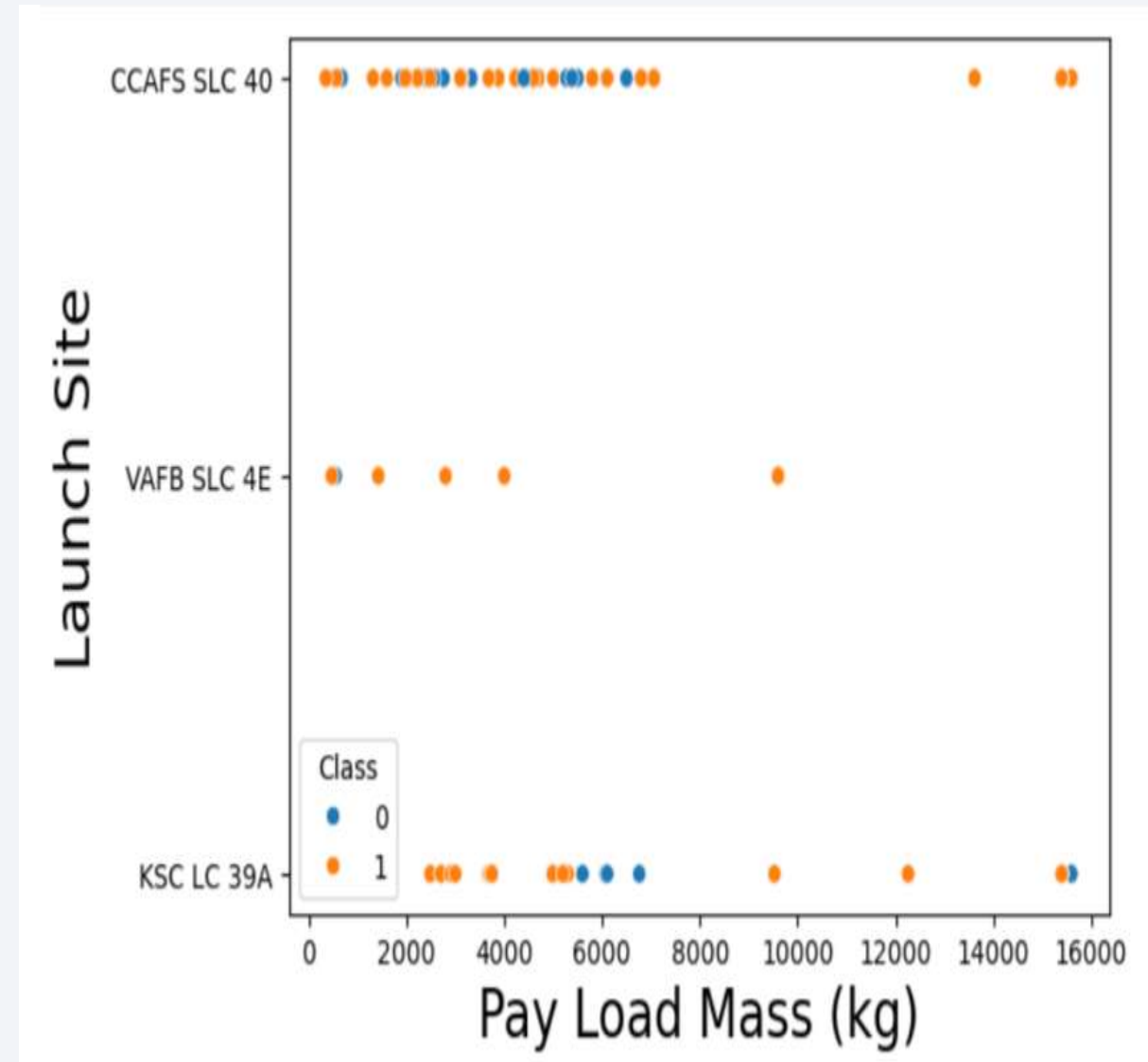
Flight Number vs. Launch Site



- Top Site Performance: CCSFS SLC-40 dominates recent operational charts with the highest volume of successful liftoffs
- Regional Distribution: Reliable launch capabilities span both the east coast (CCSFS/KSC) and the west coast (VAFB)
- Macro Trend: The historical data exhibits a clear upward trajectory, proving that overall launch success rates have steadily improved over time.

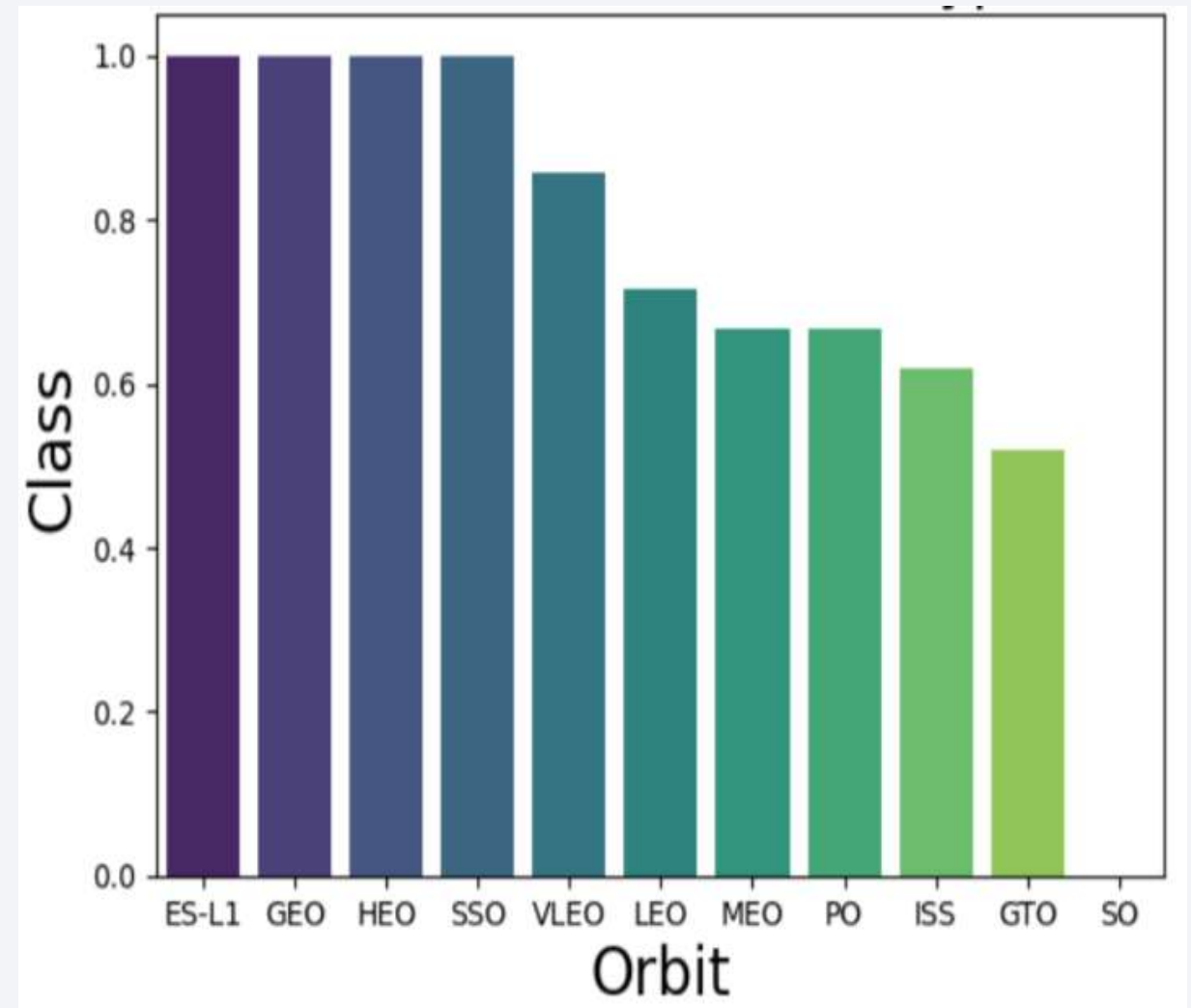
Payload vs. Launch Site

- Mass Reliability: Heavy-lift payloads above 9,000 kg maintain an excellent track record of successful orbital insertions. Infrastructure
- Constraints: Super-heavy missions over 12,000 kg rely entirely on Cape Canaveral and Kennedy Space Center facilities.
- Geographic Monopoly: The highest-capacity payload capabilities are concentrated solely on the Florida coast.

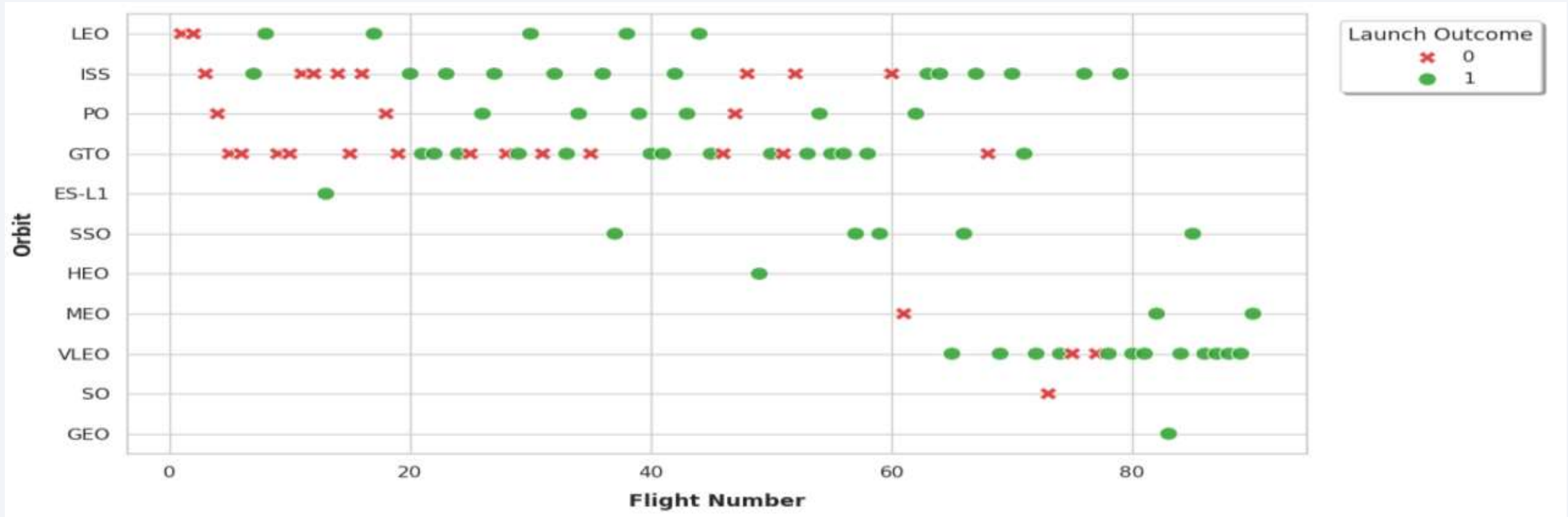


Success Rate vs. Orbit Type

- Peak Performance Orbits: ES-L1, GEO, HEO, and SSO lead the data with perfect or near-perfect launch outcomes.
- Highly Reliable Tiers: Very Low Earth Orbits (VLEO) demonstrate strong resilience with an active success rate exceeding 80%.
- Standard Operational Baselines: Low Earth Orbits (LEO) remain highly stable and efficient, securing a success rate above 70%.



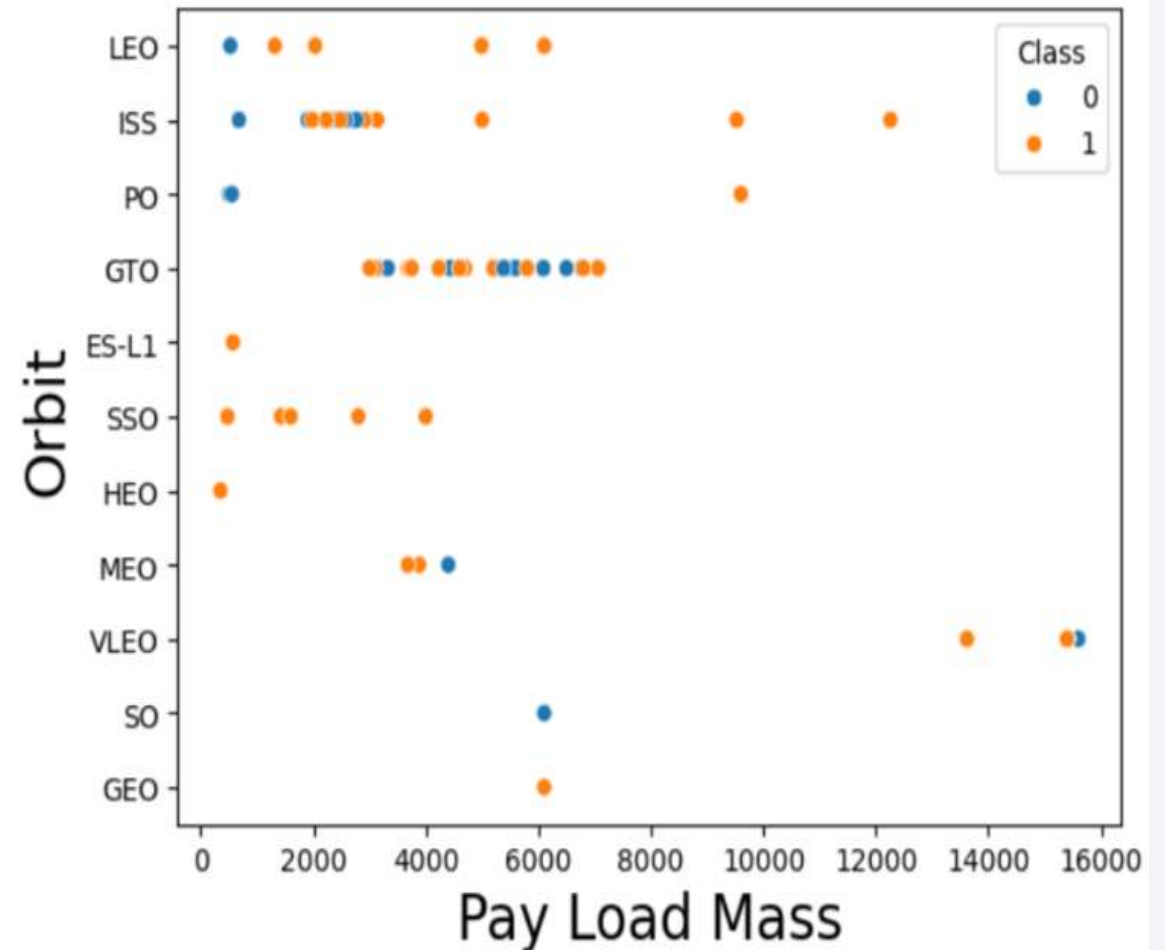
Flight Number vs. Orbit Type



- LEO Flight Correlation: Low Earth Orbit demonstrates an experience curve effect, where a higher volume of flights directly corresponds to a higher success rate.
- GTO Performance Independence: Geostationary Transfer Orbit outcomes remain independent of flight numbers, suggesting that operational maturity or flight sequence does not dictate success in this orbit.
- VLEO Business Potential: Very Low Earth Orbit (VLEO) stands out as a highly lucrative commercial frontier, heavily supported by a recent surge in flight frequency and high overall reliability.

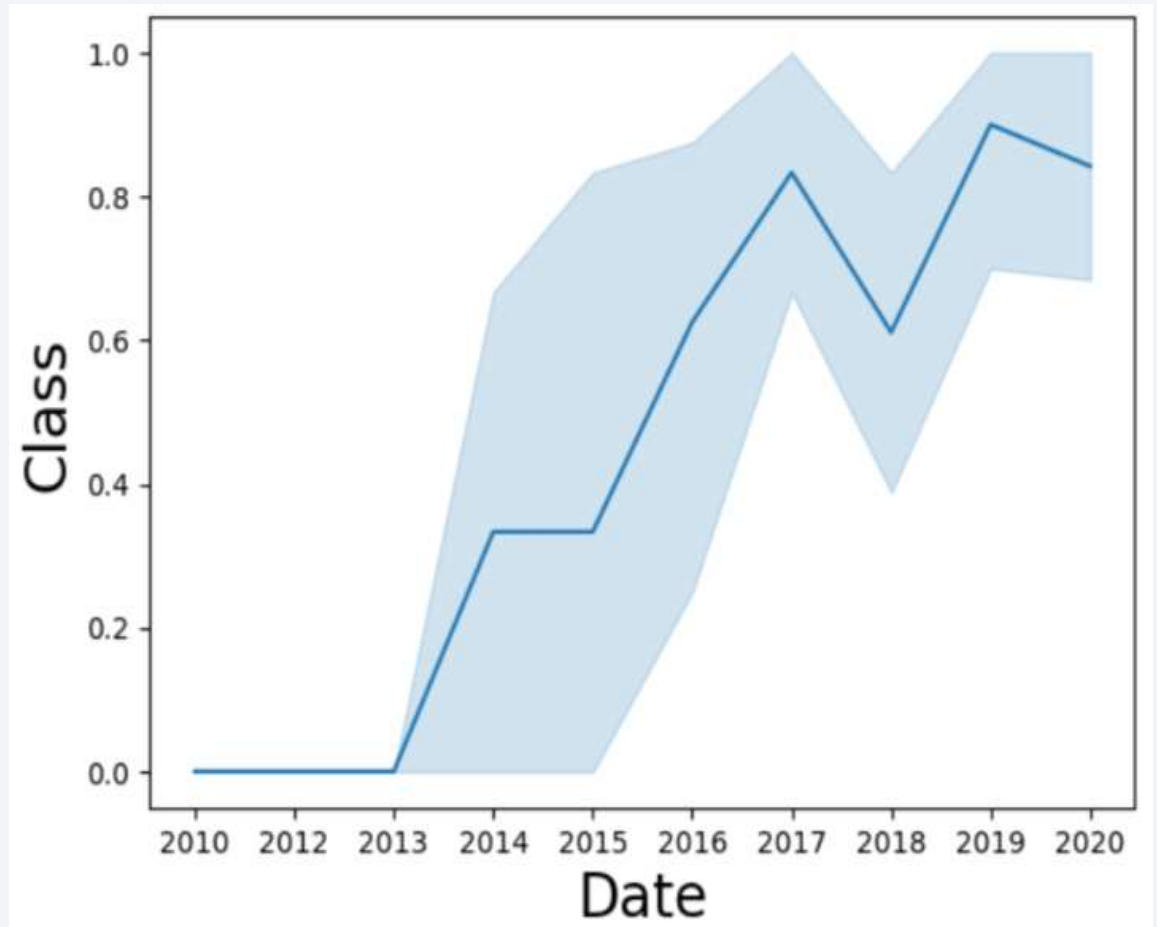
Payload vs. Orbit Type

- GTO Mass Independence: Geostationary Transfer Orbit success rates appear completely decoupled from payload mass variations.
- ISS Versatility: The International Space Station serves as a highly robust orbital destination, handling a massive spectrum of payload weights with strong operational reliability.
- Low-Frequency Orbits: SSO and GEO represent niche target profiles in this dataset, characterized by a very low volume of overall liftoffs.



Launch Success Yearly Trend

- Sustained Growth Trajectory: Following the initial phase, launch reliability achieved a consistent, year-over-year upward trend leading up to 2020.
- Developmental Baseline: The first three years served as a critical learning period focused on resolving early engineering bottlenecks and stabilizing flight systems.
- Operational Maturity: The post-2013 surge reflects the successful standardization of reusable rocketry and optimized flight controls.



All Launch Site Names

- By applying the DISTINCT SQL keyword to filter the SpaceX dataset, we isolated and identified the company's unique operational launch sites. The query confirms that the data tracks exactly four primary launch facilities.

```
[29]: %sql select distinct Launch_Site from SPACEXTABLE;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[29]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

Launch Site Names Begin with 'CCA'

- The SQL query displayed below filters the SpaceX dataset to return the first five records where the launch site name begins with the prefix "CCA".

```
[30]: %sql select * from SPACEXTABLE WHERE Launch_Site like 'CCA%' limit 5;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[30]:
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

- By executing the SQL query below, we aggregated the data to determine that the total payload mass carried by boosters on missions for NASA equals exactly 45,596 kg.

```
[32]: %sql select SUM (PAYLOAD_MASS_KG_) from SPACEXTABLE where Customer = 'NASA (CRS)';  
* sqlite:///my_data1.db  
Done.  
[32]: SUM (PAYLOAD_MASS_KG_)  
45596
```

Average Payload Mass by F9 v1.1

- By executing the SQL query below, we calculated that the average payload mass carried by the "F9 v1.1" booster version is exactly 2,928.4 kg.

```
[56]: %sql select AVG (PAYLOAD_MASS_KG_) from SPACEXTABLE where Booster_Version like 'F9 v1.1';  
      * sqlite:///my_data1.db  
Done.  
[56]: AVG (PAYLOAD_MASS_KG_)  
      2928.4
```

First Successful Ground Landing Date

- Based on the historical dataset, the first successful ground pad landing occurred on December 22, 2015. This milestone was achieved during the historic Falcon 9 Flight 20 mission at Cape Canaveral's Landing Zone 1 (LZ-1).

```
[57]: %sql select min(date) from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)';  
      * sqlite:///my_data1.db  
Done.  
[57]: min(date)  
      2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- By executing the SQL query below, we isolated the specific booster missions that successfully landed on an ocean drone ship while carrying a payload mass between 4,000 kg and 6,000 kg.

```
[25]: %%sql
select Booster_Version
FROM SPACEXTABLE
WHERE Landing_Outcome = 'Success (drone ship)'
and PAYLOAD_MASS__KG_ >4000
and PAYLOAD_MASS__KG_ < 6000;
```

```
* sqlite:///my_data1.db
Done.
```

```
[25]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

- By applying the wildcard % character within a CASE WHEN statement, we successfully standardized and consolidated all variations of successful and failed mission outcomes. Grouping the dataset by these unified classifications and calculating the total records for each group produced the clear summary below.

```
[19]: %%sql
      select
      case
        when Mission_Outcome like 'Success%' then 'Success'
        else 'Failure'
      end as Consolidated_Outcome,
      count(*) as Total_Count from SPACE_TABLE group by Consolidated_Outcome;
```

* sqlite:///my_data1.db

Done.

```
[19]: Consolidated_Outcome  Total_Count
```

Failure	1
Success	100

Boosters Carried Maximum Payload

- By implementing a subquery with the MAX() function within the WHERE clause, we successfully isolated the specific booster versions that carried the maximum payload mass registered in the dataset. This dual-layered filter approach allowed the database engine to dynamically identify the absolute peak weight threshold before extracting all matching mission records.

```
%%sql
SELECT
  (SELECT Count(*) FROM SPACEXTABLE AS t2
   WHERE t2.Booster_Version <= t1.Booster_Version
    and t2.PAYLOAD_MASS_KG_=(SELECT MAX (PAYLOAD_MASS_KG_) FROM SPACEXTABLE)
  ) AS List_Number,
  t1.Booster_Version,
  t1.PAYLOAD_MASS_KG_
from SPACEXTABLE AS t1
where t1.PAYLOAD_MASS_KG_=(
  select MAX (PAYLOAD_MASS_KG_)
  FROM SPACEXTABLE
)
order by t1.Booster_Version ASC;
```

* sqlite:///my_data1.db
Done.

List_Number	Booster_Version	PAYLOAD_MASS_KG_
1	F9 B5 B1048.4	15600
2	F9 B5 B1048.5	15600
3	F9 B5 B1049.4	15600
4	F9 B5 B1049.5	15600
5	F9 B5 B1049.7	15600
6	F9 B5 B1051.3	15600
7	F9 B5 B1051.4	15600
8	F9 B5 B1051.6	15600
9	F9 B5 B1056.4	15600
10	F9 B5 B1058.3	15600
11	F9 B5 B1060.2	15600
12	F9 B5 B1060.3	15600

2015 Launch Records

- By applying a combination of the WHERE clause alongside LIKE, AND, and BETWEEN operators, we successfully isolated failed drone ship landings from the year 2015. The query precisely extracts the specific booster versions, launch site names, and exact failure dates matching these historical parameters.

```
sqlite
select
  case substr(Date,6,2)
    when '01' then 'January' when '02' then 'February' when '03' then 'March' when '04' then 'April'
    when '05' then 'May' when '06' then 'June' when '07' then 'July' when '08' then 'August'
    when '09' then 'September' when '10' then 'October' when '11' then 'November' when '12' then 'December'
  end as Month_name,
  Landing_Outcome,
  Booster_Version,
  Launch_Site
from SPACEXTABLE
where Landing_Outcome like 'Failure (drone ship)' and substr(Date,0,5) = '2015';
```

* sqlite:///my_data1.db

Done.

Month_name	Landing_Outcome	Booster_Version	Launch_Site
January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- We retrieved the landing outcomes and their total counts within the 2010–2017 timeframe. By grouping the outcomes and ordering them in descending fashion, we generated a ranked list of the most frequent results.

```
[59]: %%sql
select
    ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS Row_Num,
    Landing_Outcome, COUNT(*) as Outcome_Count
from SPACEXTABLE
WHERE date between '2010-06-04' and '2017-03-20'
group by Landing_Outcome
order by Outcome_Count desc;
```

* sqlite:///my_data1.db

Done.

```
[59]:
```

Row_Num	Landing_Outcome	Outcome_Count
1	No attempt	10
2	Success (drone ship)	5
3	Failure (drone ship)	5
4	Success (ground pad)	3
5	Controlled (ocean)	3
6	Uncontrolled (ocean)	2
7	Failure (parachute)	2
8	Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and the glowing city lights of the Eastern United States and parts of Canada at night. The background is the deep blue of the atmosphere and the black of space.

Section 3

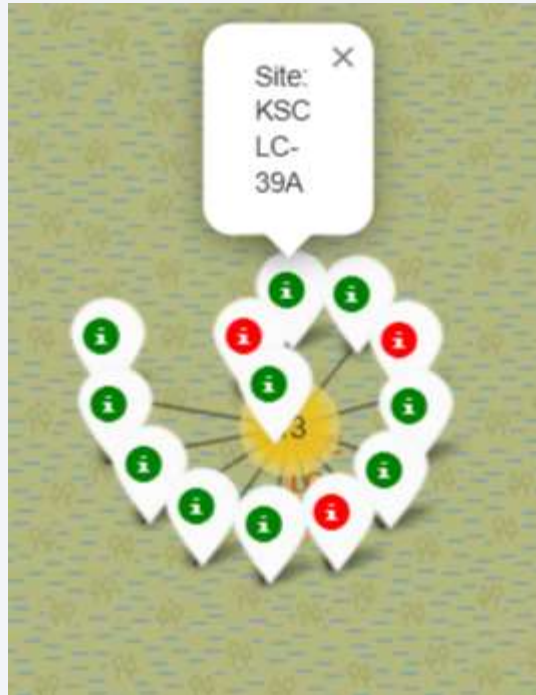
Launch Sites Proximities Analysis

All Launch Sites

- SpaceX launch sites are located on the coasts of Florida and California. They are positioned close to highways and railways to facilitate transportation.



Color-Coded SpaceX Launch Site Markers



Florida launch sites color-coded by outcome: green markers indicate success, and red markers indicate failure.

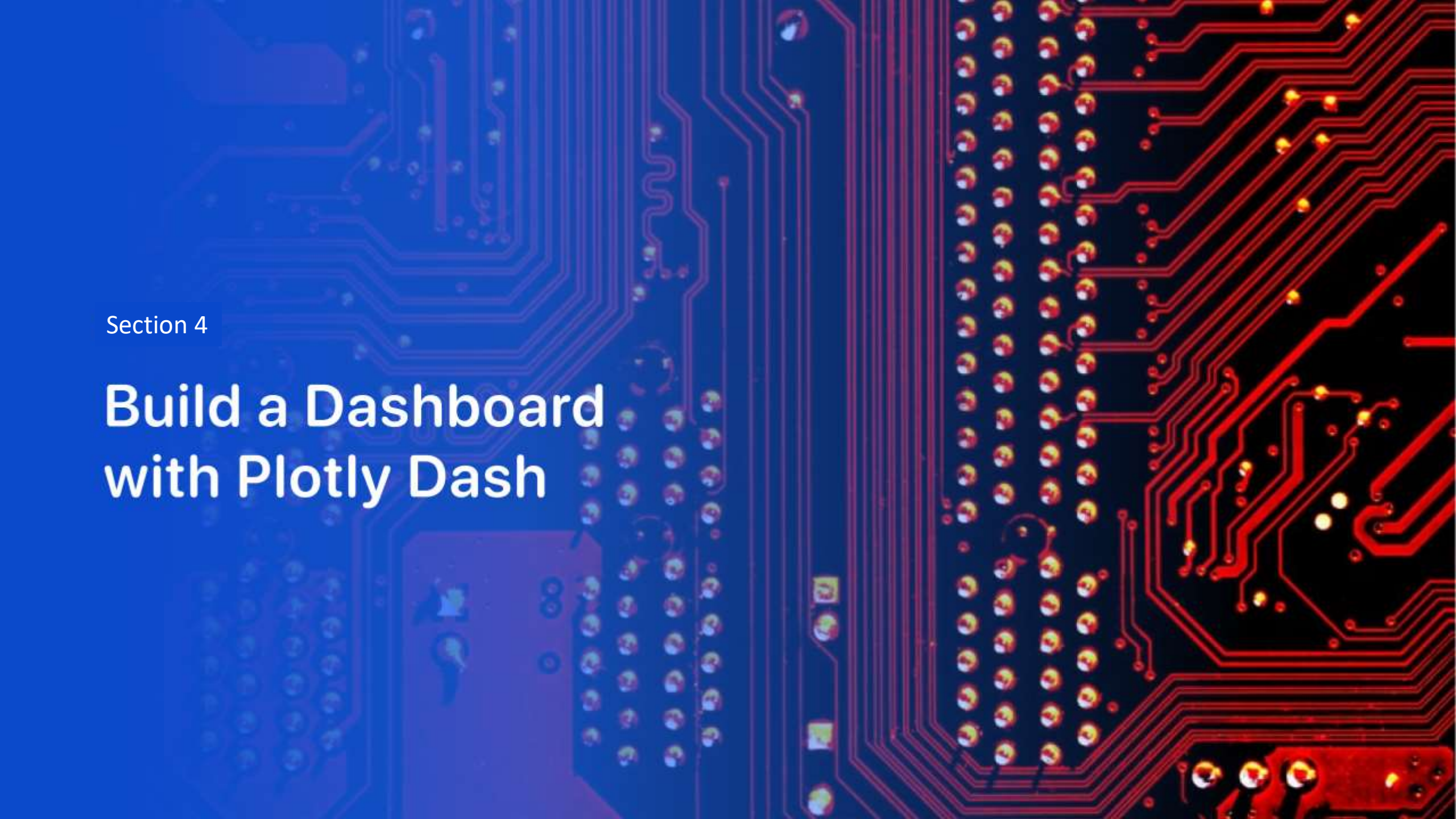


California Launch Site

Launch Sites Proximity to Infrastructure

- The CCAFS SLC-40 and LC-40 launch sites feature ideal logistics: they are close to transportation networks (roads and railways) and far from residential areas.





Section 4

Build a Dashboard with Plotly Dash

Pie Chart Analysis: Launch Success Percentages by Site

- KSC LC-39A: The Most Successful Launch Site
- The Data: Holds the highest total launch volume and success rate in the dataset.
- Why It Excels:
 - Legacy NASA infrastructure allows for faster turnaround times between launches.
 - Geographically ideal for orbital insertions over the Atlantic Ocean.
 - Seamless integration with surrounding Kennedy Space Center logistics.

Total Success Launches for All Launch Sites

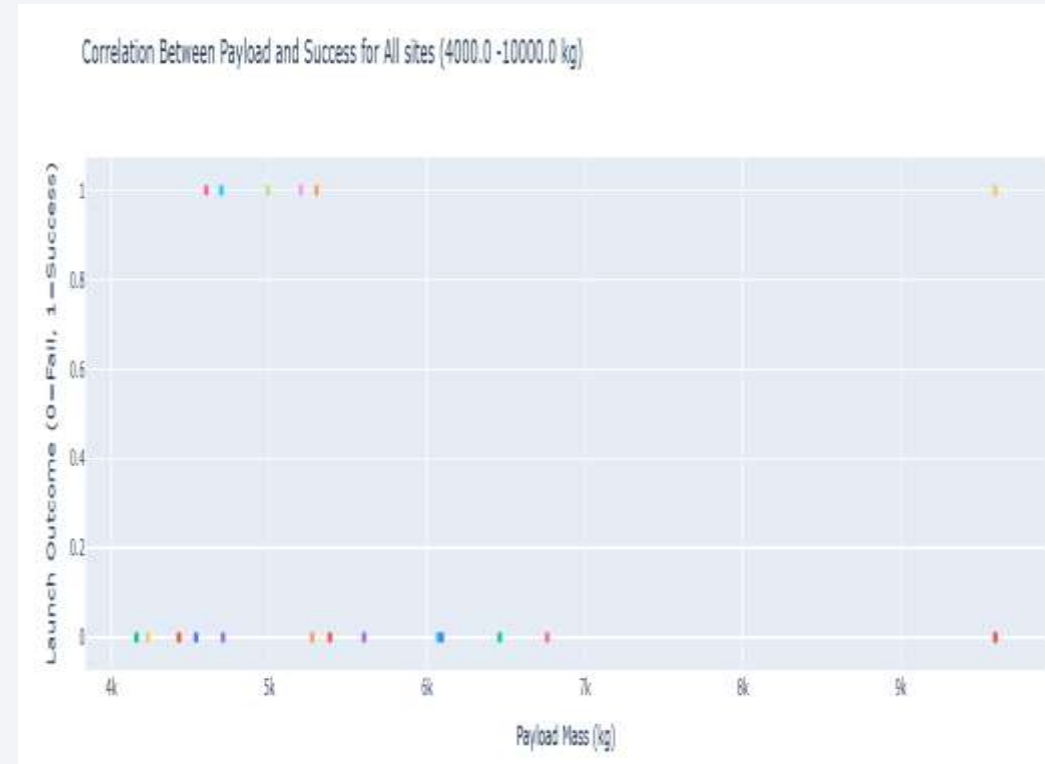
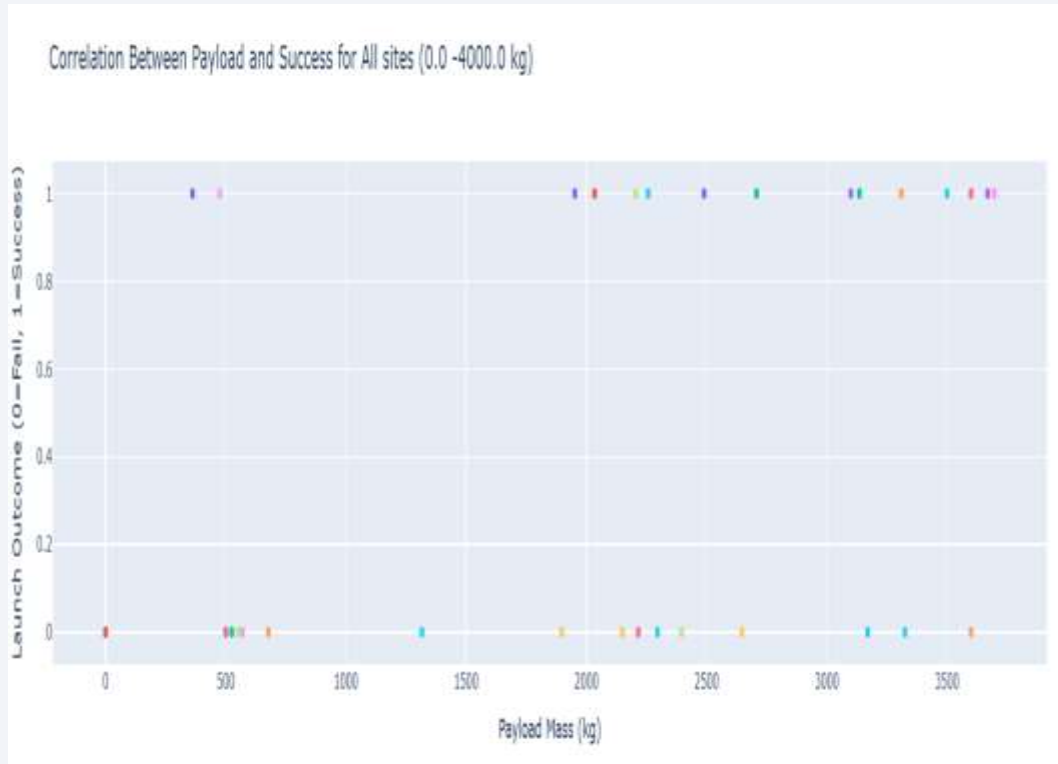


<Dashboard Screenshot 2>

- Replace <Dashboard screenshot 2> title with an appropriate title
- Show the screenshot of the piechart for the launch site with highest launch success ratio
- Explain the important elements and findings on the screenshot

Payload Mass vs. Launch Outcome Filtered by Selected Range

- Mission outcomes strongly favor lighter payloads, which show a distinct performance advantage over heavier cargo configurations.





Section 5

Predictive Analysis (Classification)

Classification Accuracy

- The Decision Tree Classifier emerged as the top-performing model with the highest overall accuracy.

```
best_method = max(scores, key=scores.get)
print (f'\n the best performing method is: {best_method} with a score of {scores [best_method]}')
if best_method=='Decision Tree':
    print('Best params is:', tree_cv.best_params_)

if best_method=='Logistic Regression':
    print('Best params is:', logreg_cv.best_params_)

if best_method=='SVM':
    print('Best params is:', svm_cv.best_params_)

if best_method=='KNN':
    print('Best params is:', knn_cv.best_params_)
```

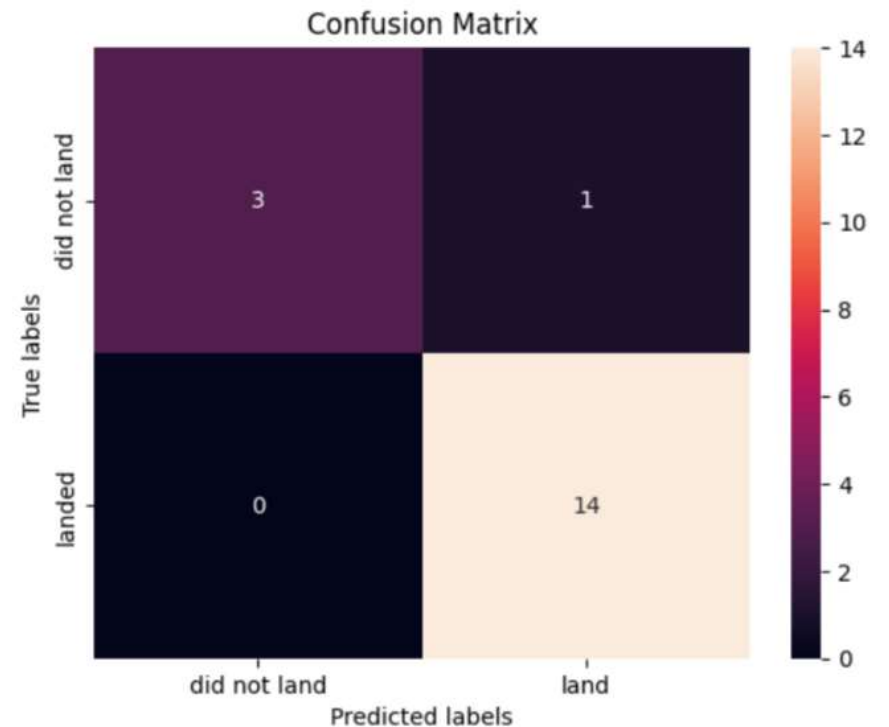
```
Decision Tree:0.9444444444444444
Logistic Regression:0.9444444444444444
SVM:0.8888888888888888
KNN:0.9444444444444444
```

```
the best performing method is: Decision Tree with a score of 0.9444444444444444
Best params is: {'criterion': 'gini', 'max_depth': 2, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, 'splitter': 'random'}
```

Confusion Matrix

- This slide presents the Confusion Matrix for our evaluated Decision Tree Classifier on 18 hidden test records. The visual demonstrates a highly optimized model with a test classification accuracy of 94.4%.
- Crucially, the model achieved a Type II error rate of zero, meaning it successfully captured 100% of all actual successful landings. Furthermore, it did not fall victim to class imbalance; it correctly isolated 3 out of the 4 landing failures in our test subset, yielding only a single False Positive.
- This strong clustering along the diagonal axis confirms that our predictive pipeline is stable, mathematically valid, and directly correlates physical features like payload mass and launch pads to real-world orbital landing outcomes.

```
] yhat = tree_cv.predict(X_test)  
plot_confusion_matrix(Y_test,yhat)
```



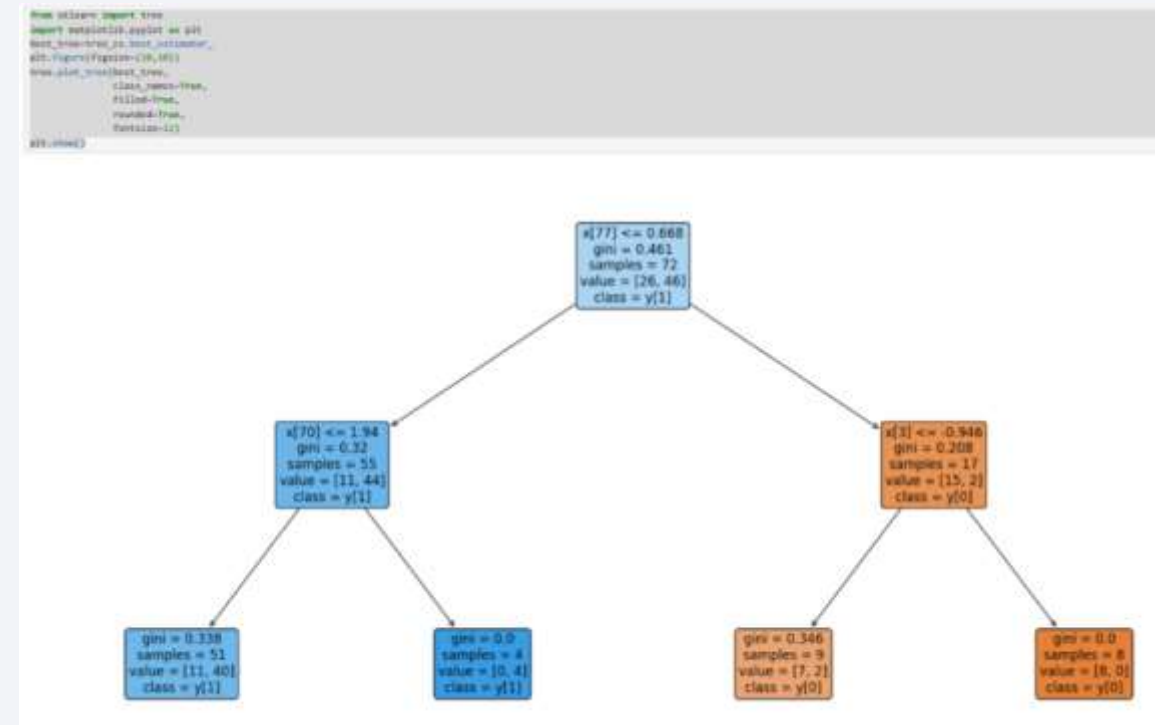
Conclusions

➤ Key Findings:

- Volume Drives Success: Higher flight volumes at a launch site correlate directly with increased success rates.
- Positive Trend: Launch success rates grew steadily from 2013 through 2020. Top Orbits: EL-L1, GEO, HEO, SSO, and VLEO achieved the highest success rates.
- Leading Site: KSC LC-39A recorded the highest number of successful launches.
- Best Model: The Decision Tree classifier performed best for this machine learning task.

Appendix

- Plotting the Decision Tree allows us to visually detect overfitting by identifying overly deep or hyper-specific branches.
- Why this model is NOT overfitted:
 - Shallow Depth: The tree is only two levels deep, which prevents it from learning overly complex rules.
 - Large Node Samples: Three out of four leaf nodes hold large, significant chunks of data (51, 9, and 8 samples).
 - No Single-Sample Leaves: None of the terminal nodes are split down to a single outlier data point (samples = 1).
 - Healthy Gini Purity: The pure leaves (gini = 0.0) are achieved cleanly with reasonable group sizes (4 and 8), rather than forcing forced splits on noisy data



Thank you!

